OS & SP

Name : M Ramya Sree **Roll No :** 2520030094

SKILL – 23

1.Skill23.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <time.h>

#define JOBS 4

int main()
{
    pid_t pids[JOBS];
```

```
printf("=====================================\n");
printf(" OSSP PROGRAM 23\n");
printf(" Large Pipeline & Performance Test\n");
printf("=====================================\n\n");

printf("Launching %d jobs...\n\n", JOBS);
fflush(stdout);

/* Launch multiple jobs */
for (int i = 0; i < JOBS; i++) {

    pids[i] = fork();

    if (pids[i] < 0) {
        perror("fork");
        return 1;
    }
```

```
    /* Child process */
    if (pids[i] == 0) {

        printf("Job %d started | PID = %d\n",
               i + 1, getpid());
        fflush(stdout);

        /* Simulate different workloads */
        sleep(i + 1);

        printf("Job %d completed | PID = %d\n",
               i + 1, getpid());
        fflush(stdout);

        exit(0);
    }
}
```

```
/* Parent waits for all jobs */
printf("\nParent waiting for all jobs...\n\n");

for (int i = 0; i < JOBS; i++) {

    int status;

    if (waitpid(pids[i], &status, 0) < 0) {
        perror("waitpid");
        continue;
    }

    if (WIFEXITED(status)) {

        printf("Job %d finished successfully "
               "with status %d\n",
               i + 1, WEXITSTATUS(status));
```

```
        } else if (WIFSIGNALED(status)) {


            printf("Job %d failed with signal %d\n",
                   i + 1, WTERMSIG(status));
        }
    }


    printf("\n====================================\n");
    printf("All jobs have been processed.\n");
    printf("Performance test completed.\n");
    printf("====================================\n");


    return 0;
}
```

OUTPUT :

